Week 44

v1.0.1

Table of contents

1 Sequence Proporties 1
2 Fasta til DataFrame 1
3 Self-study: Calculations in python 2
4 Self-study: Units in python 4

```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[pyodide] @ https://au-mbg.github.io/fysisk-biokemi/
wheels/fysisk_biokemi-0.2.2-py3-none-any.whl"
```

```
import fysisk_biokemi.widgets as fsb_widgets
```

1 Sequence Proporties

Run the codecell below and copy paste in the sequence of your protein. Then use the widget to adjust the pH and see how the charge of your protein changes.

```
fsb_widgets.sequence_properties()
```

2 Fasta til `DataFrame`

The widget below accepts "fasta" files as input and you can then use the slider to adjust the pH and see how the charge of your protein changes.

Fasta-files is a specific type of text file and you will see that most of the time when you download a Protein, RNA or DNA sequence they will be downloaded as fasta files. Each sequence begins with a header line starting with >, which contains an identifier and optionally a description, followed by the sequence itself using the appropriate nucleotide or amino acid symbols. A FASTA file can contain one or multiple sequences, making it useful for sequence analysis, database searches, and comparing biological sequences using bioinformatics tools.

Upload your fasta file to jupyter and enter the name of the file in the codecell below:

```
from fysisk_biokemi.widgets.sequence_properties import sequence_dataframe

file_name = ""
widget = sequence_dataframe(file_name)
```

You can extract the dataframe with your current pH adjustment by using the widget.get_dataframe() below.

```
df = widget.get_dataframe()
```

---

3 Self-study: Calculations in python

```
import numpy as np
```

In the following, we will go through the some common mathematical manupulations of number, arrays and data frames using python. You should have covered most of these in "Introduction to the Molecules of Life", so we will not go through them in class.

(a) Simple calculations

Evaluate the following expressions using Python

- 10.5×4
- $5 + 10 + 15 + 12$
- $70 - 30 + 2$
- $420/10$

```
... # Replace ... with your code
...
...
...
```

> Tip
>
> Recall that the basic arithemetic operations are, in Python, represented as follows
>
> - $+ \Rightarrow$ +
> - $- \Rightarrow$ -
> - $/ \Rightarrow$ /
> - $\times \Rightarrow$ *
> - $x^y \Rightarrow$ x**y

(b) Array calculations

The ability to perform operations on all elements (or data points) is incredibly useful when doing data analysis, for example when converting units. This is enabled my array calculations.

The cell below defines an array a with all the numbers from 0 to 9.

For each entry a_i in the array calculate the expression $b_i = a_i/3 + a_i^2$ and assign that to a new array b. use the fact that you can do elementwise operations with an array - do not calculate it seperately for every element of the array.

```
a = np.arange(0, 10)
```

```
b = ...
print(b)
```

Tip

With an array all the basic arithemtic operations can do be done elementwise, so the expression $x^2 + 1$ can be done for all elements in an array as such

```
x = np.arange(0, 10)
y = x**2 + 1
```

Just like if `x` was just a single number. Similarly, expressions be involve adding arrays together like

```
y = x**2 + x
```

(c) Variables

Variables are an essential part of a program. Calculate the following where each intermediate is assigned to a variable

$$a = 5 \times 3$$
$$b = a + 9$$
$$c = \frac{4}{3}a + b - 2$$

```
a = ...
b = ...
...
print(a, b, c)
```

Bonus question: Why does `c` print as a decimal number when the others print without decimals?

(d) Parentheses

Parentheses are important, a misplaced or lacking parentheses can drastically change a calculation.

Using the variables `a`, `b` and `c` you calculate in the previous exercise, evaluate the following expressions

- $a \times (b + c)$
- $a \times b + c$
- $\frac{a+b+c}{b+c}$
- $(b - a)\left(\frac{1}{c} + 1\right)$
- $(b - a)\left(\frac{1}{c+1} + 1\right)$

```
q1 = ...
q2 = ...
q3 = ...
q4 = ...
```

```
q5 = ...
print(q1, q2, q3, q4, q5)
```

> Tip
>
> If you're in doubt having an extra set of unecessary parentheses is better than missing a necessary set of parentheses. Just be careful to place them correctly.

(f) Functions

For repeated computations it's good practice to define a function.

Define a function that calculates the following expression

$$f(x) = \frac{Ax + B + C}{A + B}$$

Where $A = 10$, $B = 5$, $C = 2.5$

```
def fun_function(x):
    result = ...
    return result
```

Now evaluate the function for $x = 62.5$, $x = 629.25$, $x = 42$ and $x = 2025$

```
fun_output_1 = fun_function(...) # Your code replaces ...
fun_output_2 = ... # Same here
fun_output_3 = ... # And here
fun_output_4 = ... # And finally here.

print(f"{fun_output_1:.1f}")
print(f"{fun_output_2:.1f}")
print(f"{fun_output_3:.1f}")
print(f"{fun_output_4:.1f}")
```

> Tip
>
> The input of a function will often be the independent variable and the functions calculates the independent variable.
>
> Later when we get to regression/fitting this will be the case.

---

4 Self-study: Units in python

As you may have noticed, numbers do not have a units in python. In science, all quantities need to be reported with and associated unit - ideally SI or with an appropriate prefix. In this course, we will represent units as a

separate string variable that can be printed with the answer as shown below. This is also the format you should use for the exam.

(a) Unit as a string - examples

Length of an ideal H-bond:

```
len_Hbond = 0.28
unit_len_Hbond = "nm"

print("An ideal H-bond is", len_Hbond, unit_len_Hbond)
```

```
An ideal H-bond is 0.28 nm
```

Molecular weight of human alcohol dehydrogenase:

```
MW_ADH = 42
unit_MW_ADH = "kDa"

print("MW of ADH is", MW_ADH, unit_MW_ADH)
```

```
MW of ADH is 42 kDa
```

Approximate diameter of human liver cell:

```
diam_hlc = 30
unit_diam_hlc = "µm"

print("An typical liver cell is", diam_hlc, unit_diam_hlc, "wide")
```

```
An typical liver cell is 30 µm wide
```

(b) Unit as a string - exercises

Convert to millimolar: 0.15 M

```
conc = ...      # Replace ... with your code
unit_conc = "..."

print("The concentration is", conc, unit_conc)
```

```
The concentration is Ellipsis ...
```

Convert to Å: 13.2 nm

```
length = ...      # Replace ... with your code
unit_length = "..."
```

```
print("13.2 nm corresponds to", length, unit_length)
```

```
13.2 nm corresponds to Ellipsis ...
```
